

Puppeteer: A Network Planner for AI Training Workloads

Paper #454, 12 pages body, 14 pages total

Abstract

The explosive growth of large-scale AI training has pushed traditional data-centre networking to extreme scale. Current architectures, designed for the bursty unpredictability of cloud tenants, struggle to accommodate the massive, synchronous bandwidth demands of modern deep learning. In this paper, we argue that the deterministic nature of AI workloads allows us to abandon reactive congestion control in favour of proactive, precision orchestration.

We present Puppeteer, a centralised network planner that completely rethinks the networking stack for AI. Puppeteer analyses the deep structure of the training job, including model architecture, parallelisation strategies, and collective algorithms, to synthesise a contention-free execution plan before the first packet is sent. This approach intelligently prioritises critical-path communication to maximise training velocity. Our evaluation demonstrates that Puppeteer is a practical leap forward. It reduces the training time of GPT models by up to 6% and cuts exposed communication time by 33% compared to standard ECMP and DCQCN stacks. Most critically, Puppeteer achieves a *zero-queue* invariant and entirely eliminates network congestion even in the presence of runtime variability. By replacing the stochastic reaction with deterministic planning, Puppeteer offers a blueprint for the next generation of congestion-free AI infrastructure.

1 Introduction

The rapid scaling of modern AI training workloads places unprecedented pressure on the networking infrastructure that underpins today’s data-centres. Training state-of-the-art models involves tens and hundreds of thousands of accelerators exchanging terabytes of data per second, pushing networks to operate at the edge of their bandwidth, latency, and reliability limits. Unlike traditional cloud workloads, which are dominated by short-lived, latency-sensitive request–response traffic [20, 27] or loosely synchronised bulk transfers [5], AI training introduces substantially different communication workloads and requirements. These workloads depend on highly structured, repetitive communication patterns with tight synchronisation requirements. Collective operations such as all-reduce and all-to-all induce sustained, incast-heavy traffic, expose tail-latency sensitivity at millisecond timescales, and amplify the impact of congestion, packet loss, and stragglers, as any inefficiency

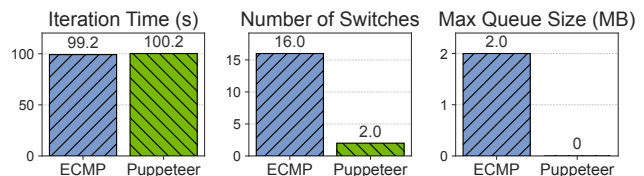


Figure 1: For a GPT-3 training iteration, Puppeteer squeezes out the same performance as ECMP while using 8× fewer switches, all with a *zero-queue* guarantee.

is directly translated to wasted expensive compute resources given the synchronous nature of the workload [3].

Acknowledging these emerging patterns, there is a series of works in the networking community that revisit traditional networking mechanisms specifically through the lens of AI workloads. Specifically, this line of research focuses on routing [3, 6], congestion control [28, 29], optical circuit switching [9, 14, 30], quality of service [8], network scheduling [34], and collective communication [4, 32].

In this work, we argue that previous work has approached networking for AI in a very *myopic* way, providing point solutions while maintaining the same architecture and mechanisms of the networking stack. We believe that the properties of AI workloads allow for a fundamental rethink and rearchitecting of the networking infrastructure specifically for AI training. Given the workload predictability, we advocate co-optimising the routing, congestion control, and quality of service subsystems of the networking stack in a centralised architecture where a coordinator has the full overview and control of every communication on the fabric. This coordinator can plan in advance and proactively enforce its plan entirely at the endpoints, instead of depending on sophisticated reactive in-network mechanisms.

Based on this insight, we design and implement Puppeteer, a centralised network planner for AI training workloads. Puppeteer takes the description of AI training jobs as input (in the form of a Chakra execution trace [33]) and produces a precise network plan based on a greedy algorithm. This plan controls the mapping of flows to network routes and their network bandwidth at different stages during AI training. Puppeteer can enforce this plan entirely at end-hosts based on mechanisms and protocols that are already available in modern NICs.

Such a centralised approach can aggressively simplify the networking stack, improve performance, and lead to a more stable and predictable infrastructure for AI training workloads. First, Puppeteer increases the network utilisation by taking advantage of all available network paths *without* the operational complexity of packet spraying. Second, Puppeteer eliminates the need for a congestion control mechanism by carefully scheduling and limiting the rate at the end-host, leading to a *zero* queue network, while also potentially reducing the hardware complexity for future NICs and switches. Third, Puppeteer can carefully prioritise network traffic in such a way to reduce the overall training time, without depending on network priorities implemented in network switches.

Despite its benefits, this centralised approach faces several open challenges and poses practical deployment concerns. More specifically, the network planner should be able to produce a plan within a reasonable time even for large-scale jobs. This requirement is becoming increasingly critical as the scale of single AI training jobs continues to grow rapidly, with state-of-the-art models now requiring tens to hundreds of thousands of GPUs [36], necessitating multi-site data-centres due to power provisioning and other constraints [7].

Unlike prior proposals [26], Puppeteer operates at coarser granularity, *i.e.*, at the level of training iterations; thus, it does not need to deal with individual packets or short flows. Additionally, Puppeteer focuses only on inter-node communication within the scale-out network, which commonly only hosts the communication related to data and pipeline parallelism [21]; unlike expert parallelism, this communication is highly predictable.

In addition to scalability, Puppeteer’s plan should be robust to variability in both the workload itself and the underlying infrastructure. Our detailed workload analysis shows that the majority of runtime variabilities can be *synchronised by the training job itself* due to the tight coupling of communication and computation phases. Puppeteer also introduces *resource dependencies* to synchronise communications and address the rare cases where runtime variability can break the validity of the plan.

Motivated by the growing importance of efficient large-scale infrastructure for recent AI models, we gear our focus toward single-job planning. Using network simulation, we demonstrate that Puppeteer can improve training time by up to 6%, *i.e.*, equivalent to tens and even hundreds of hours in large-scale training; see Section 6.1.

Additionally, Puppeteer can achieve the same training time with $8\times$ **fewer switches** in comparison to the ECMP + DCQCN stack employed in clusters today, as shown in Figure 1. Furthermore, this is achieved with a zero-queue guarantee, while remaining completely unaffected by extreme variation and stragglers.

Contributions. In this paper, we:

- Design and implement Puppeteer, a centralised network planner for AI training workloads that assign route and rate to flows while co-optimising routing, congestion control, and quality of service.
- Present a traffic engineering policy that prioritises AI traffic based on its stage within the training job, reducing exposed communication, and thereby minimising overall training time.
- Evaluate Puppeteer using network simulations that use accurate AI workloads and model infrastructure variability & straggler behaviour, providing more realistic experiments while remaining simulator-based.

2 AI Workload Characteristics

This section identifies some unique characteristics of AI workloads that distinguish them from general-purpose data-centre traffic. We highlight our insights using ♡.

Flow sparsity. Training workloads exhibit orders of magnitude fewer flows than traditional web services, reducing the decision space for scheduling.

Sparse connectivity. The communication matrix is sparse, meaning that at any given time a GPU communicates with only a small subset of peers, minimising local contention.

Periodicity. The iterative nature of gradient-based training implies that scheduling decisions can be reused over thousands of iterations, amortising the cost of planning.

Deployment. AI training typically use two different networks: (i) high-bandwidth intra-node scale-up domain (*e.g.*, NVLink) and (ii) inter-node scale-out domain (*e.g.*, Ethernet). Distributed training relies on a combination of parallelisation strategies, each with distinct communication patterns.

Data Parallelism (DP) replicates the entire model across GPUs to parallelise input processing. It requires an all-reduce operation at the end of each iteration to synchronise parameters. This communication typically occurs across the inter-node scale-out domain.

Tensor Parallelism (TP) shards the model “horizontally” (intra-layer) for models larger than a single GPU. This requires blocking all-reduce operations after every layer. Due to the extremely high volume and latency sensitivity of this traffic, TP is almost exclusively confined to the high-bandwidth, intra-node scale-up domain.

Pipeline Parallelism (PP) shards the model “vertically” (inter-layer) across GPUs. This requires point-to-point (P2P) communication between pipeline stages. Various pipe-lining schedules exist [10, 16, 21], but all involve predictable P2P transfers that often traverse the inter-node scale-out domain.

Expert Parallelism (EP) is used for Mixture-of-Experts (MoE) models. Similar to TP, the model is sharded horizontally,

but by “experts” rather than strict layer partitions. This requires variable all-to-all collectives to route tokens to the appropriate experts. As token routing is not known until runtime, expert parallelism introduces data-dependent and unpredictable communication. Leading-edge clusters, such as those based on NVIDIA NVL72 [23] or the upcoming NVL144 [24], feature massive single-node domains (72 or 144 GPUs) connected by high-bandwidth scale-up domains. Since most MoE models use a number of experts that fit within these node limits [13], highly variable expert-routing traffic remains largely intra-node.

💡 In existing deployments, inter-node communication in scale-out domain stays deterministic and predictable.

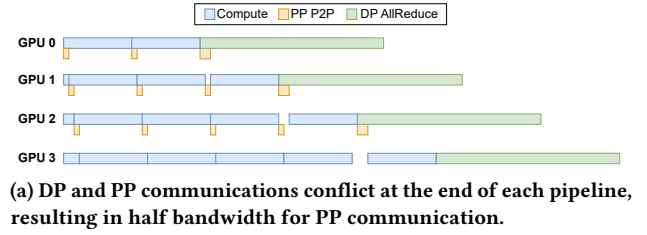
Workload structure. Training workloads can be represented as a Directed Acyclic Graph (DAG), which contains all the information necessary to estimate precise compute runtimes and identify critical path dependencies. Additionally, recent works [31, 35] have developed AI workload synthesisers that allow modelling performance variation within the DAG. Chakra execution traces [33] are one of the common ways to represent AI workloads as a DAG.

Communication dependency. By analysing the DAG, we can identify dependency between different communications, which enables us to prioritise flows that lie on the critical path. Figure 2a illustrates a common scenario where DP and PP communication overlap at the end of a pipeline stage. Each GPU must perform two transfers: a P2P transfer to the next pipeline stage and a DP all-reduce to sync gradients.

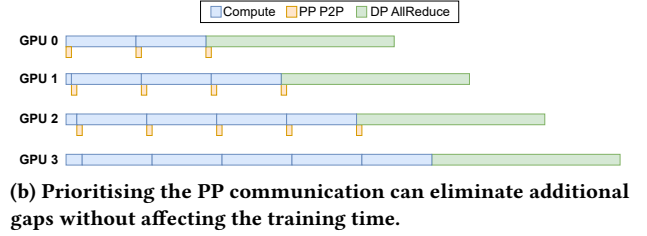
Since the final pipeline stage must complete its backward pass before DP all-reduce can finalise, all-reduce operations of earlier pipeline stages have slack, *i.e.*, increasing their completion time does not increase the iteration time (see Figure 2b). In contrast, the P2P pipeline transfer is strictly blocking. Therefore, prioritising the P2P transfer over the DP all-reduce can reduce the effective step time *without* altering the training semantics.

💡 Not all communication is equally critical.

Real-world variability. Real-world performance is never perfectly static. A recent study by Alibaba [15] revealed that jobs waste approximately 10.4% of GPU hours due to stragglers, with the root cause almost entirely being computation variability rather than network issues. Therefore, any robust network plan must be resilient to compute jitter. Fortunately, we observe that the AI training workloads are naturally *self-synchronising*. For example, as shown in Figure 3, if a GPU starts computation late (or straggles), its peers in the TP group must wait to complete the synchronous TP all-reduce. This effectively synchronises the entire TP group with the slowest member. This delay propagates to



(a) DP and PP communications conflict at the end of each pipeline, resulting in half bandwidth for PP communication.



(b) Prioritising the PP communication can eliminate additional gaps without affecting the training time.

Figure 2: Prioritising PP P2P intra-job communication reduces training time, whereas DP AllReduce can be delayed without impacting training time.

the pipeline stage and eventually to the entire DP group. This tight coupling ironically simplifies planning: since the workload naturally realigns itself at every synchronisation point, small drifts do not accumulate into chaotic divergence, allowing static plans to remain valid even in the presence of minor runtime variations. In addition to stragglers, the variable all-to-all communication of MoE models within the scale-up domain can also be simply modelled as variance in computation time.

💡 AI training jobs are naturally *self-synchronising*.

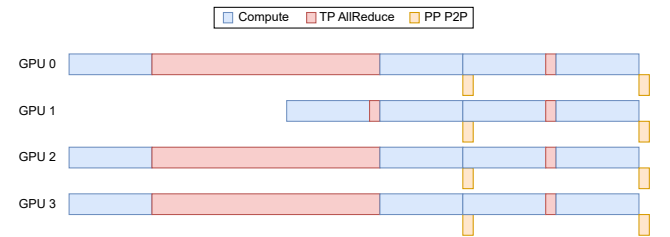


Figure 3: Due to intra-layer dependencies with TP, training jobs synchronise themselves even when a GPU is straggling.

3 Existing Approaches

While the distinct characteristics of AI training workloads have prompted a resurgence in specialised networking research, existing approaches typically rely on reactive, in-network mechanisms or isolate optimisation to specific layers of the stack. This section reviews these approaches.

Improving routing. The standard approach to traffic distribution in data-centre networks, Equal Cost Multi-Path (ECMP), relies on flow hashing to statistically balance traffic across available links. While effective for traditional workloads with high flow concurrency, ECMP falters under the specific characteristics of AI training, which generates a sparse communication matrix dominated by few, high-bandwidth “elephant” flows. This low entropy often leads to hash collisions and persistent congestion on specific links, even while others remain idle. Although recent large-scale deployments have attempted to mitigate this by including connection information in the hash calculation [8], such measures are incomplete; relying on probabilistic load balancing for deterministic workloads fundamentally precludes ideal network utilisation.

Packet spraying has emerged as a prominent alternative to flow-based hashing [1, 12], theoretically achieving optimal load balancing by distributing traffic at the granularity of individual packets. However, this optimality comes at the cost of significant implementation complexity. Packet spraying necessitates the re-ordering of out-of-order packets at the receiver, imposing substantial memory and processing overheads on the NIC, only supported by latest hardware. Furthermore, the effect of Head-of-Line (HOL) blocking is exacerbated on this approach, as packet re-ordering is further complicated. These practical limitations have stifled the adoption of packet spraying in production environments, leaving operators to rely on sub-optimal flow-based routing.

Collective optimisation. In the academic sphere, several works have proposed sophisticated solvers to compute optimal routing schemes for individual collectives [4, 17, 18, 32]. These solutions frame routing as an optimisation problem and apply Integer Linear Programming (ILP) to find optimal solutions. While these solvers can produce theoretically optimal schedules, they are typically NP-Hard and computationally prohibitive for online use. For instance, SyCCL [4] reports runtimes of hours to calculate routing tables for a single collective across a modest number of GPUs. While they can be applied to every collective in a job, taking hours per collective for 32 GPUs is not scalable. Furthermore, they do not take into account application-level and inter-collective optimisations such as prioritising one communication step over another.

Inter-job optimisation. Significant effort has also been directed toward optimising inter-job network conflict. Solutions range from simple heuristics, such as offsetting periodic communication phases to avoid temporal overlap [28], to explicit prioritisation schemes based on job-level metrics like GPU utilisation [3]. For example, Crux assigns priorities at a coarse, job-level granularity. While effective at reducing macroscopic interference, this approach misses the nuance of intra-job dependencies: not all communication within a

high-priority job is equally critical. Moreover, these works primarily serve to avoid conflict scenarios rather than resolving them efficiently when they inevitably occur.

Intra-job optimisation. Conversely, relatively little work has focused on intra-job optimisation. Real-world deployments still largely use basic ECMP routing and barely explore advanced congestion control options [8]. Recent academic proposals like Hermod [34] use microbatch IDs and layer information to assign discrete priority levels to flows, but such discrete prioritisation is difficult to generalise across different model architectures and pipe-lining strategies. EchelonFlow [25] attempts to minimise tail flow completion times for grouped communication but relies on an NP-hard formulation. Similarly, a recent route planning effort [39] shows promise but often operates within the constraints of existing protocols. Ultimately, these works attempt to optimise the network within current system boundaries. As mentioned earlier in Section 1, we focus on intra-job optimisations given the increasing importance of large-scale infrastructure for modern AI models. However, unlike these efforts, we argue that the unique characteristics of AI training allow for an entirely new approach - one where the entire stack is co-optimised.

4 Puppeteer: A Network Planner for AI Training

Puppeteer is a centralised network planning system designed to leverage the full-stack visibility of AI training clusters. Unlike traditional control planes that react to traffic dynamics, Puppeteer operates offline, synthesising a precise execution schedule for a single AI training job before the first gradient is exchanged. To tackle performance variations that can lead to divergence from the plan, Puppeteer introduces *resource dependencies*, a novel mechanism that informs GPUs when to synchronise at runtime.

High-level design. The planner ingests three primary inputs to construct its world view (see Figure 4): (i) the AI workload representation (e.g., a Chakra execution trace) containing compute kernels, message sizes, and dependencies; (ii) the physical job distribution, mapping logical ranks to physical GPU locations within the network topology; and (iii) the specific collective implementation (e.g., ring-based or tree-based algorithms) used by the communication library.

By combining these inputs, Puppeteer processes the training step to identify contention points and resolves them algorithmically rather than relying on runtime packet arbitration. The output of this synthesis is a comprehensive execution plan, distributed to every participating GPU at the start of the training session. This plan dictates the behaviour

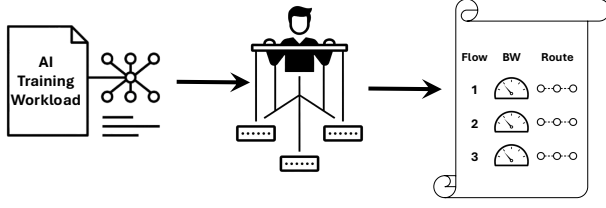


Figure 4: Puppeteer generates a plan composed of a route and rate for every flow within an AI training job.

of every inter-node flow with strict granularity. Specifically, for each communication operation, the plan prescribes:

- ① A specific network route, pinning flows to paths that maximise bisection bandwidth usage while avoiding collisions.
- ② A bandwidth pacing schedule, defining the precise transmission rate at specific intervals to ensure link capacity is never exceeded.
- ③ A set of dependency notifications, identifying which peer GPUs must be synchronised before transmission begins to mitigate the impact of runtime jitter (further detailed in Section 4.1).

Because the iterative nature of training means that communication patterns repeat identically across epochs, this plan is generated once and reused for every training iteration, amortising the planning cost.

A significant characteristic of this design is the elimination of network contention. Since all flows are explicitly scheduled to fit within the available bandwidth and routed to avoid overlap, queue build-up is theoretically eliminated. Consequently, Puppeteer removes the need for traditional, reactive congestion control protocols; the network fabric acts purely as a dumb pipe, while intelligence and arbitration are enforced proactively at the endpoints.

4.1 Network Resource Allocation

① **Route assignment.** A practical network planner for AI workloads must strike a balance between route optimality and calculation speed. The routing algorithm must be sufficiently lightweight (as opposed to ILP optimisers like TA-CCL [32]), and it should exploit the deterministic nature of the workload to outperform oblivious mechanisms like ECMP and achieve ideal load balancing similar to packet spraying. Puppeteer addresses these requirements by employing a greedy bucketing approach inspired by discrete event simulation. As AI training traffic is sparse, we can feasibly track and allocate individual flows manually. Puppeteer iterates through the workload DAG chronologically, resolves dependencies, and calculates the precise start time of every communication kernel. When a collective node is reached,

Puppeteer decomposes it into its constituent point-to-point flows according to the user-specified implementation (e.g., Ring-AllReduce).

Maintaining state. Puppeteer uses two primary data structures to maintain network state:

- A map of link $\rightarrow$ list[active_flows], allowing $O(1)$ updates to bandwidth sharing when flows enter or leave a link.
- A map of switch $\rightarrow$ map[count $\rightarrow$ list[links]], which buckets outgoing links by the number of active flows they currently carry.

This second structure enables constant-time selection of the outgoing link with the minimum number of active flows at each hop.

Assignment logic. The planning logic is driven by two primary events:

- (1) **Flow start.** When a flow is ready to start, Puppeteer determines the communication scope based on the physical topology (e.g., a 3-layer Clos network):
 - *Intra-node.* The flow stays within the server. Puppeteer calculates completion time based on fixed scale-up fabric bandwidth without consuming scale-out resources.
 - *Intra-pod.* The flow traverses up to the Top-of-Rack (ToR) & Spine switches and back down.
 - *Inter-pod.* The flow must traverse the full hierarchy, reaching the Super-Spine (Core) layer before descending to the destination pod.
- (2) **Flow completion.** When a flow completes, its occupied resources are released. Puppeteer network state and recalculates bandwidth for any remaining flows sharing the same link, potentially accelerating their completion.

Based on this scope, the flow is routed hop by hop through the layers. At each switch, the greedy logic selects the least-utilised link. Once the path is set, bandwidth is allocated: if the selected links are idle, the flow receives full line rate; otherwise, bandwidth is shared among concurrent flows. Additionally, Puppeteer updates the expected completion of *all* pre-existing flows on the selected path to reflect the reduced bandwidth. Algorithm 1 summarises Puppeteer’s route assignment logic.

Complexity and scalability. This greedy approach is highly efficient. For a job with F total flows and a network diameter of D , the time complexity is roughly $O(F \cdot D \cdot C)$, where C represents the constant overhead of hash-map updates and re-calculating bandwidth for colliding flows. Given the sparsity of AI workloads, the collision factor C remains small. Space complexity is $O(L + F_{active})$, where L is the number of links and F_{active} is the maximum number of concurrent flows. This efficiency allows Puppeteer to generate plans for large-scale clusters in seconds, a

Algorithm 1 Puppeteer’s route assignment algorithm.

Require: src_gpu, dst_gpu
determine src_node, dst_node
if $src_node = dst_node$ **then**
 return intra-node path
end if
select $u_0 \leftarrow$ least-active uplink at ToR
if $src_pod = dst_pod$ **then**
 select $d_0 \leftarrow$ least-active downlink
 return intra-pod path
end if
select $u_1 \leftarrow$ least-active uplink
select $d_1 \leftarrow$ least-active downlink
for all flows using selected link **do**
 update bandwidth and expected completion time
end for

significant improvement over ILP-based methods. Finally, we note that Puppeteer is designed with modularity in mind; while the greedy algorithm is the default, the routing engine is decoupled from the enforcement mechanism, allowing operators to plug in offline solvers (like an ILP) if strictly optimal routing is required for specific static workloads.

② **Bandwidth allocation.** Once routes are assigned, the next logical step is bandwidth provisioning. In a contention-free scenario, every flow is allocated the maximum rate available along its path (e.g., the line rate). However, when multiple flows share a physical link, bandwidth must be arbitrated. The default policy in most networks is *max-min fairness*, where each competing flow receives an equal slice of the link’s capacity. While fair in a generic sense, this oblivious policy is suboptimal for AI training workloads. As discussed in Section 2, not all communication within a training step is equally critical; delaying a non-blocking background transfer to accelerate a blocking gradient exchange can significantly improve job completion time (JCT). This raises a key challenge: how can we algorithmically quantify the “criticality” of each flow to inform bandwidth allocation, while maintaining the scalability of Puppeteer’s logic?

Communication prioritisation. Our approach relies on the insight that minimising individual Flow Completion Time (FCT) does not necessarily minimise global JCT. Instead, JCT is driven by *exposed communication time*, i.e., the duration during which computation resources are stalled waiting for data. To capture this, we introduce a new metric, **Time-to-Exposed (TTE)**, which quantifies the slack available to a communication operation before it becomes a bottleneck. Formally, it is the time difference between the completion of a communication kernel and the latest

possible start time of its dependent computation. A TTE of *zero* indicates that the flow is on the *critical* path; i.e., any delay in this flow directly delays the subsequent computation and, by extension, the entire training iteration. Conversely, a positive TTE implies that the flow can be throttled or delayed without impacting the overall timeline, provided it finishes before its slack is exhausted.

Calculating TTE. Calculating TTE presents a circular dependency: TTE depends on flow finish times, which depend on bandwidth allocation, which we intend to drive using TTE. Solving this exactly would require a complex ILP formulation. To preserve scalability, we estimate TTE using a single, optimistic simulation pass over the workload DAG. In this pass, we traverse the graph chronologically, assuming *infinite bandwidth* (or maximum line rate) for all communication. This reveals the “structural” critical path of the job, thus identifying flows that are blocking purely due to dependencies, independent of network congestion.

Using the start and finish times from this ideal execution, we calculate TTE for every communication node. For a given flow f and its dependent child node c , TTE is derived as:

$$TTE(f) = Start(c) - Finish(f)$$

where $Start(c)$ is determined by the latest finishing parent of c . If $Start(c)$ is gated immediately by $Finish(f)$, then $TTE(f) = 0$. If c is gated by another, slower parent (e.g., a long compute kernel), then $TTE(f) > 0$, representing the budget we have to de-prioritise f . Figure 5 illustrates an example of calculating TTE.

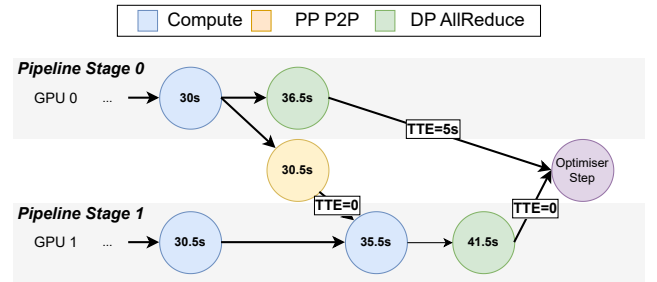


Figure 5: How TTE is calculated from a workload DAG. Each node is marked by their estimated finish time. The DP allreduce of GPU 0 has 5 s of wiggle room because the optimiser step is blocked by the DP allreduce of GPU 1.

Application-aware prioritisation. With TTE values assigned, Puppeteer’s bandwidth allocator can make informed decisions. When allocating bandwidth on a congested link, we prioritise flows with $TTE \approx 0$ over those with large positive TTEs. This strategy is particularly effective for hybrid parallelism. Consider the scenario in Figure 2, where

a GPU must perform both a Pipeline Parallel (PP) point-to-point transfer and a Data Parallel (DP) all-reduce simultaneously.

- **PP transfer.** Its dependent child is the forward pass of the next pipeline stage. In our ideal simulation, this compute cannot start until the transfer arrives. Thus, $TTE = 0$.
- **DP All-Reduce.** Its dependent child is the optimiser step at the very end of the iteration. The optimiser step is gated by the slowest DP all-reduce in the system (typically from the last pipeline stage). For early pipeline stages, the local DP all-reduce finishes well before the global synchronisation point. Thus, its TTE is large (often hundreds of milliseconds).

By inspecting TTE values, Puppeteer correctly identifies that the PP transfer is critical. Consequently, it allocates full bandwidth to the PP flow, effectively serialising the operations without penalty, while the DP flow utilises the remaining slack. This transforms network planning from a generic packet-shuffling exercise into an application-aware optimisation that directly minimises training time. Algorithm 2 summarises the TTE computation logic.

Algorithm 2 Puppeteer’s TTE computation logic.

```

for flow do
  child  $\leftarrow$  flow.child
   $t_{\text{latest}} \leftarrow 0$ 
  for  $p \in \text{child.parents}$  do
     $t_{\text{latest}} \leftarrow \max(t_{\text{latest}}, p.t_{\text{finish}})$ 
  end for
   $\text{child}.t_{\text{start}} \leftarrow t_{\text{latest}}$ 
   $TTE \leftarrow \text{child}.t_{\text{start}} - \text{flow}.t_{\text{finish}}$ 
end for
```

③ **Handling runtime uncertainty.** As discussed in Section 2, real-world data-centre environments are never perfectly static; performance jitter, OS noise, and hardware stragglers inevitably introduce variance into execution timelines. Variation can even simply be a representation of inaccurate estimation of compute time. While Puppeteer’s plan is robust due to the macroscopic self-synchronisation of training jobs, microscopic deviations pose a significant risk to a zero-queue architecture. In a network operating at 400 Gbps or higher, timing precision is critical: a mere 1 millisecond of unsynchronised transmission where a sender blasts data onto a link before a competing flow has ramped down can result in over 50 MiB of buffering. Such transient bursts can rapidly overflow switch buffers, triggering PFC pauses, and the very congestion we aim to avoid. Therefore, relying solely on a static time-based schedule is insufficient; we require a runtime mechanism to enforce the plan’s integrity even when clocks drift.

To bridge the gap between static planning and dynamic execution, Puppeteer augments the execution plan with explicit dependency barriers. During the routing phase, Puppeteer identifies not just the path for each flow, but also the specific set of “peer flows” that share physical links with it. These relationships are encoded directly into the plan distributed to the GPUs. At runtime, before initiating any communication kernel or changing transmission rates, a GPU must consult this dependency list. If a flow is scheduled to potentially share bandwidth with a peer on another host, the source GPU initiates a lightweight handshake with the corresponding peer GPU. This handshake serves as a mutual confirmation: “I am ready to transmit at rate R , are you ready to accommodate this?” Only when the peer acknowledges that it has reached the corresponding state in its own execution plan does the data transmission proceed. Note that this handshake is only necessary when the concurrent flows are *not* inherently dependent by the workload itself.

This mechanism effectively enforces a “synchronise-then-send” discipline. If a straggler delays one part of the job, the handshake ensures that aggressive senders do not flood the network prematurely. Instead, they wait at the barrier, preserving the network’s zero-queue property at the cost of slight idle time - the idle time being equivalent to the RTT, which is in the order of microseconds in modern data-centres. Such a trade-off is vastly preferable to the catastrophic latency of packet loss.

We note that these handshake packets are not themselves scheduled in the original bandwidth plan. However, because they consist of single, minimum-size frames, their impact on link utilisation is negligible. Even if they encounter transient queuing, the volume is insufficient to cause disruption, ensuring that the control plane remains responsive while the data plane remains congestion-free.

5 Practical Deployment Considerations

Puppeteer is designed with deployment practicality as a primary constraint. Rather than requiring bespoke hardware or a complete overhaul of the network control plane, Puppeteer is designed to operate as a lightweight overlay that integrates into existing AI training stacks. This section elaborates on feasibility of deploying Puppeteer in real-world settings.

Plan-once, run-forever. The Puppeteer lifecycle begins immediately after job scheduling but prior to execution. The planner consumes the workload DAG and topology graph to synthesise the execution plan, which is then serialised and distributed to all participating GPU workers during the initialisation phase. Due to the strict periodicity of AI training, this plan is generated once and cached. It is

then replayed for every subsequent training iteration. This model ensures that the planning overhead is amortised. Furthermore, the system is stateless across iterations; even if transient faults disrupt a specific step, the subsequent iteration resets to the known-good state defined by the static plan, ensuring long-term stability without complex error recovery logic.

Routing enforcement. A key challenge in centralised planning is enforcing specific paths in networks designed for oblivious routing. We propose two mechanisms to realise Puppeteer’s routing decisions:

- **Source-based routing.** Modern network hardware increasingly support source-routing headers (*e.g.*, via specific IPv6 routing extension headers such as SRv6 or encapsulation protocols). Additionally, recent RDMA transport proposals [1, 12] propose per-packet adaptive load balancing schemes, which can be exploited to enforce a path that a packet traverse in the network.
- **Deterministic ECMP.** In legacy fabrics restricted to ECMP, path pinning can be effectively achieved by controlling the hash inputs [37]. By manipulating the flow entropy fields (such as the UDP source port or flow label) in the transport header, we can theoretically deterministically target specific ECMP buckets at each hop. This would allow Puppeteer to “steer” flows onto specific physical links using standard hardware features.

Rate enforcement. To enforce the precise bandwidth pacing dictated by the plan, the hardware rate-limiting capabilities of modern NICs could be leveraged. For example, NVIDIA NICs offer various programmable features (*e.g.*, PCC [22]) that allows network developers to control rate, as done by SCR [38]. Puppeteer can rely on similar features to enforce transmission rates.

Resource dependencies. The resource dependency mechanism described in Section 4.1 is implemented by augmenting the workload DAG itself. This is currently done to enable the functionality in discrete network simulation. Rather than having to modify the lower-level transport drivers, our solution inserts synthetic dependency edges into the execution graph. These edges represent the required “handshake” operations. When the execution engine (*e.g.*, the PyTorch or TensorFlow runtime) encounters these dependencies, it would automatically trigger a lightweight blocking receive operation for the handshake packet, just as it would any other communication step. Only upon receipt of this confirmation does the engine proceed to launch the subsequent communication kernel. This approach allows us to implement complex synchronisation logic entirely in user-space software, requiring no changes to the NIC firmware or the underlying collective communication primitives.

6 Evaluation

This section evaluates the efficacy of Puppeteer through detailed network simulations. Our evaluation focuses on three primary aspects: (i) performance improvements, (ii) robustness to uncertainties, and (iii) planning scalability.

Although our primary goal is single-job performance, we briefly evaluate Puppeteer in multi-job scenarios.

Implementation. We implement Puppeteer’s planner in Python in thousands of LOC, and extend Astra-Sim, a cycle-accurate distributed learning simulator, with additional features to support route & rate enforcement and handling runtime synchronisation packets during network simulations. We also add packet spraying support by distributing packets in a round-robin style across each available link.

Workload. We simulate a workload consisting of training a GPT-3 model (175B parameters) on a cluster of 32 and 256 GPUs. We mainly present results for the 32-GPU cluster (due to reduced simulation time), but our takeaways are still relevant for the larger cluster. The workload employs standard 3D parallelism: Tensor Parallelism ($TP = 4$) within nodes, Pipeline Parallelism ($PP = 2$) with a 1F1B schedule, and Data Parallelism ($DP = 4$). This workload was generated using MLSynth [31], a tool for generating accurate synthetic AI workload traces as a Chakra execution trace. This ensures our workload accurately characterises the dependencies observed in AI training workloads. The physical topology simulates a multi-pod, 3-layer Clos network with lossless 100-Gbps links, as depicted in Figure 6. Consistent with modern deployment practices, TP communication is confined to the high-bandwidth intra-node scale-up fabric, while PP and DP communication traverses the inter-node scale-out fabric. Unless otherwise stated, we use a standard Ring All-Reduce implementation for collectives. To explore the impact of network capacity, we vary the number of spine switches to simulate both over-subscribed and non-blocking scenarios.

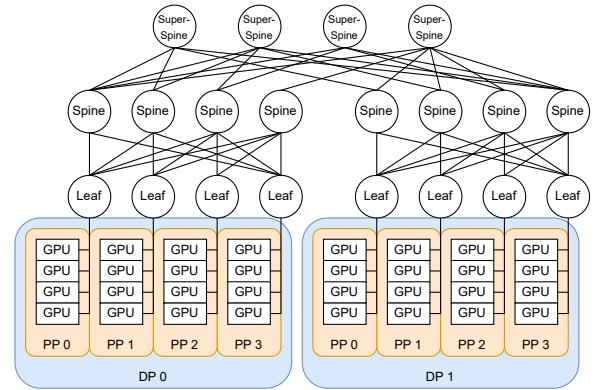


Figure 6: Topology used for evaluation.

6.1 Does Puppeteer Improve Performance?

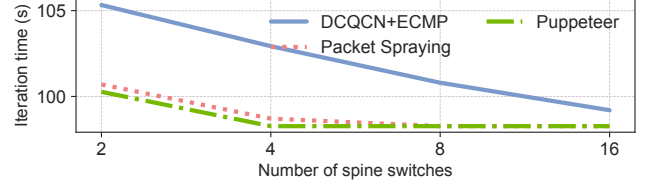
We first evaluate Puppeteer under ideal conditions (*i.e.*, assuming deterministic compute times) to establish an upper-bound for performance improvements. We compare Puppeteer against two baselines: standard ECMP routing with DCQCN congestion control, and packet spraying, which represents the theoretical limit of load balancing.

Figures 7a and 7b illustrate the iteration time and exposed communication overhead for the GPT-3 workload. In scenarios where the network is oversubscribed (*i.e.*, $\#Spine \geq 4$), ECMP suffers significantly from hash collisions, leading to sub-optimal link utilisation. Packet spraying mitigates this by perfectly distributing traffic across all available paths, yielding a performance improvement of approximately 5%.

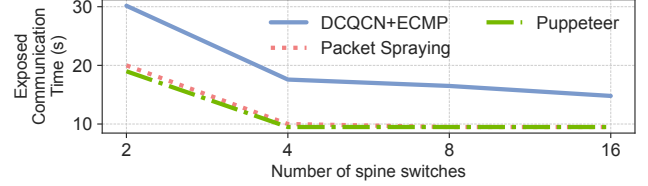
Impact of application-awareness. Puppeteer outperforms packet spraying by an additional 1%. This additional gain stems from its application-aware bandwidth allocation. At the boundary of each pipeline stage, critical PP point-to-point transfers compete with non-critical DP all-reduce operations. Packet spraying blindly attempts to service both simultaneously, causing mutual slowdown. In contrast, Puppeteer detects that the PP transfer has a TTE of zero and allocates it full bandwidth, serialising the flows.

By delaying the non-critical DP transfer, Puppeteer reduces the *exposed* communication time—the metric that directly impacts training speed—by 5% and 33% compared to ECMP and packet spraying, respectively; see Figure 7b. It is worth mentioning that without application-awareness, Puppeteer could achieve a similar result as packet spraying, *i.e.*, achieve ideal network utilisation. While 6% total improvement may appear modest, in the context of large-scale model training, it is substantial. Training GPT-3 on 1024 GPUs typically requires roughly 35 days [2, 21]. A 6% reduction translates to approximately 50.4 hours of wall-clock time. Across a 1024-GPU cluster, this saves over 51,600 GPU-hours - a massive reduction in operational cost and energy consumption.

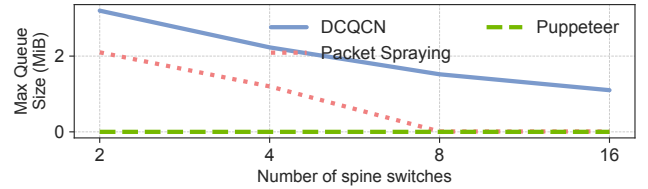
Figure 7c highlights the most distinct advantage of Puppeteer: the elimination of network buffering. Standard protocols like DCQCN react to congestion only after queues have begun to build. Consequently, ECMP suffers massive queue spikes when multiple flows start simultaneously. Due to ECMP’s probabilistic nature, it will never achieve perfect balancing, which means Puppeteer achieves the same training time as ECMP+DCQCN with an eighth of the switches. Even packet spraying, despite ideal load balancing, cannot prevent queue build-up when the aggregate demand exceeds bisection bandwidth. Puppeteer, conversely, induces *no queueing*. By enforcing rates, the network operates strictly within its capacity. This “zero-queue” property not only



(a) Iteration time.



(b) Exposed communication time.



(c) Maximum network queuing.

Figure 7: Puppeteer completely eliminates network buffering, and improves iteration & exposed communication times by prioritising critical pipeline communication, when training GPT-3.

stabilises jitter but also implies that hardware designers could potentially reduce switch buffer sizes, lowering infrastructure costs. Furthermore, because Puppeteer explicitly serialises conflicting traffic in this particular workload, it can achieve contention-free performance with fewer spine switches than oblivious schemes, effectively increasing the usable capacity of oversubscribed networks.

6.2 Is Puppeteer’s Plan Robust to Runtime Variations?

Real-world training is not deterministic. We now introduce performance variation to the simulation using MLSynth’s compute variance feature. We inject noise into compute kernels, reducing average performance by 10%, consistent with straggler studies in production clusters [15].

As illustrated in Figure 8, even small compute variances can cause communication steps to drift, potentially creating overlaps that were not present in the static plan. Without intervention, this would lead to significant congestion, as described in Section 4.1. However, Puppeteer’s runtime

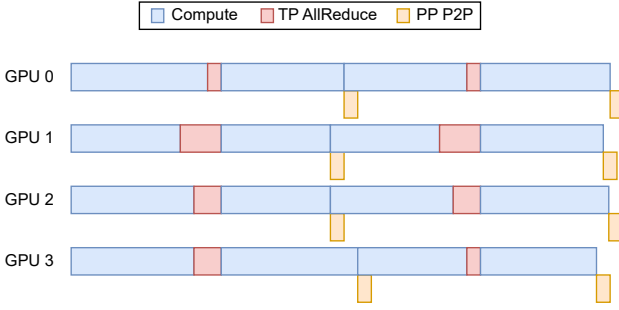


Figure 8: Communication that is supposed to occur at the same time is offset by compute variation. Puppeteer’s synchronisation mechanism is necessary to prevent queue build-up as well as to capitalise on free bandwidth.

synchronisation mechanism effectively shields the network from this fate.

Figure 9 demonstrates that while the iteration time exhibits noise corresponding to the compute variance, the relative performance gains over baselines remain consistent. The synchronisation handshakes ensure that queues do not form, trading negligible RTT-scale idle time for robustness against packet loss.

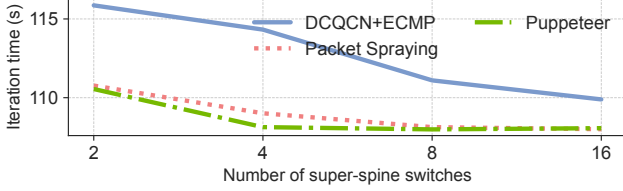


Figure 9: Puppeteer’s plan remains effective despite uncertainty from real-world compute variation of around 10%.

Stress testing. To test the limits of our design, we increase compute variation up to 30% and introduce “extreme straggler” scenarios where a single GPU lags significantly behind the group. Figure 10 confirms that Puppeteer’s planning still remains robust. The heavy-tailed straggler scenario highlights the self-synchronising nature of 3D parallelism: if one GPU straggles, the entire TP group blocks, which cascades to block the pipeline stage, effectively resetting the schedule. In the rare event that a straggler causes a “fast” node to attempt communication early, Puppeteer’s resource dependencies prevent the fast node from flooding the network. Thus, even under extreme macroscopic variations, the network plan remains safe and the zero-queue invariant is preserved.

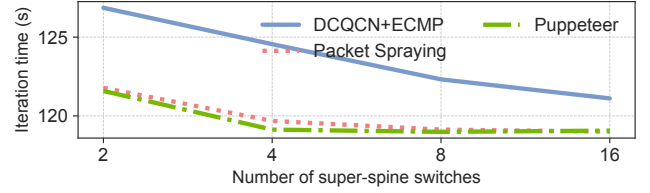


Figure 10: Despite extreme stragglers in the iteration, gains remain consistent.

6.3 Does Puppeteer Scale to Large Clusters?

A central concern for any centralised control plane is scalability: can a single entity effectively plan for thousands of nodes without becoming a bottleneck? To answer this, we evaluate the execution time of Puppeteer’s planning algorithm as a function of cluster size.

Computational C’ complexity. As detailed in Section 4.1, the runtime of the greedy algorithm scales linearly with the number of flows. The number of flows in a training job is determined by the parallelisation strategy rather than the raw GPU count (e.g., in a 3D-parallel workload, the number of concurrent P2P flows per pipeline stage depends on TP group size). Although our current implementation is single-threaded Python code, the planning for different time-steps or independent collective groups could theoretically be parallelised. Therefore, the results presented here represent a conservative baseline with significant room for optimisation. Moreover, it is possible to implement Puppeteer using other programming languages to further improve runtimes.

Resource dependencies. We quantify the communication overhead introduced by the runtime dependency mechanism. This control traffic is a function of the number of flows contending for shared physical links, which is determined by the network radix and the physical distribution of the job. In a fully non-blocking, under-subscribed network, this mechanism remains dormant because every flow receives a dedicated path. Overhead is only incurred when topological constraints force Puppeteer to schedule flows on the same link.

Table 1 details the volume of synchronisation packets generated as the cluster scales. For our baseline experiment with 32 GPUs, the cost is vanishingly small. The system performed only 58 synchronisation handshakes throughout the entire iteration, resulting in a total network-wide overhead of 12 KiB. The majority of this data consists of packet headers rather than payload. When contextualised against the hundreds of megabytes transferred during a standard Pipeline Parallel P2P step, this control traffic is effectively negligible.

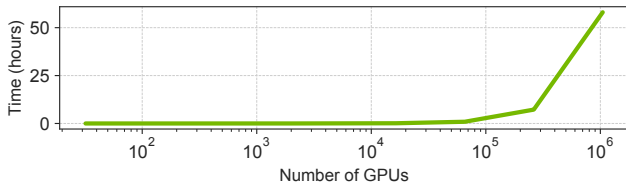
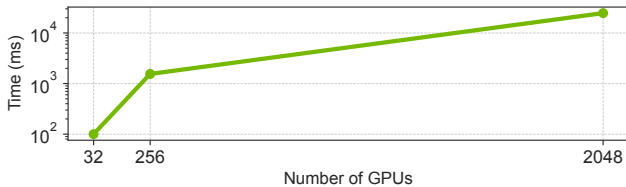
As the cluster size increases, the absolute overhead grows but remains minor relative to the aggregate throughput of

Table 1: Overhead induced by Puppeteer’s resource dependency mechanism during GPT-3 training iteration.

Num GPUs	Num Handshakes	Traffic Overhead	Overhead/GPU
32	58	12 KiB	375 B
256	580	120.1 KiB	469 B
131,072	580	72 MiB	549 B

the system. At 256 GPUs, our simulation recorded 120.1 KiB of total control traffic. Extrapolating this trend to a hyper-scale deployment of 131,072 GPUs, we estimate the total overhead would reach approximately 72 MiB per iteration across the entire data-centre. This scalability is preserved because the number of conflicts does not grow quadratically with the node count. Since the size of Tensor Parallel groups remains constant even at hyper-scale, the density of flow conflicts remains bounded, ensuring that Puppeteer’s safety mechanism never becomes a source of congestion.

Planning latency vs. Cluster size. Figure 11 plots the wall-clock time required to generate a complete network plan. For our experimental baseline of 32 GPUs, Puppeteer completes the planning phase in approximately 100 ms—effectively *instantaneous*. For a production-scale cluster of 1024 GPUs (typical for GPT-3 training), Puppeteer synthesises the full routing and bandwidth plan in approximately 12 s. We can contrast this one-time cost with the potential savings. As established in Figure 7, Puppeteer yields a conservative 6% improvement in iteration time. For a GPT-3 training run that spans 35 days, this 6% speed-up translates to a reduction of roughly 50.4 hours in end-to-end training time. The return on investment is therefore stark: we invest 12 s of pre-computation to save over 50 hours of expensive GPU cluster time.

**(a) Planning time scales up to 50 hours for 1 million GPUs.****(b) Zoomed-in view shows medium-size clusters compute in seconds.****Figure 11: Puppeteer can compute the plan in a reasonable time with the number of GPUs.**

Hyper-scale projections. Finally, we push our analysis to an extreme hyper-scale scenario of 1 million GPUs. At this magnitude, our unoptimised Puppeteer takes approximately 50 hours to generate a solution. Training runs at this magnitude would span many months. Therefore, even a 50-hour planning phase constitutes a fraction of a percent of the total job lifecycle, confirming that centralised planning is computationally tractable for even the largest foreseeable AI workloads.

6.4 Is Puppeteer Useful for Multiple Jobs?

Support for multi-job scheduling is a critical requirement for cloud providers, especially in multi-tenant settings. However, coordinating multiple independent training jobs introduces significant complexity for a centralised offline planner. Parsing multiple jobs with Puppeteer is simple - the DAGs of each job are merged and processed at the same time when generating the plan. However, unlike a single job where iteration cycles are periodic, independent jobs typically have heterogeneous step times and phase alignments. Consequently, the global traffic pattern does not repeat periodically, meaning a single static plan cannot suffice unless all jobs are forcibly synchronised to a common “super-cycle”, which would introduce wasteful idle time. Without forced synchronisation, Puppeteer would need to transition from a static planner to a dynamic “babysitter”, continuously calculating and streaming new plans in real-time as jobs drift. This fundamental shift in complexity places general multi-tenancy outside the scope of our current work.

Regardless, to demonstrate the potential value of extending our approach, we explore an isolated multi-job scenario where intra-job prioritisation still yields benefits. We simulate an iteration of three concurrent jobs sharing a cluster, arranged such that they begin at the same time, thus resulting in their DP groups contend for super-spine uplinks (see Figure 12). Each job is scaled to 90% the size of the previous one. Crucially, the phase alignment is such that the largest job is performing a non-critical all-reduce (for an early pipeline stage) while the smaller job is attempting a critical all-reduce (for its final stage), as seen in Figure 12.

Existing inter-job schedulers like Crux [3] typically assign static priority based on resource usage, blindly favouring the job with higher GPU utilisation. In contrast, Puppeteer’s TTE analysis correctly identifies that the smaller job’s communication is on the critical path, while the larger job has slack. Figure 13 demonstrates that by dynamically prioritising the smaller job’s critical traffic, Puppeteer achieves a 7% reduction in iteration time for the smaller jobs without penalising the large job, while maintaining zero-queue operation. This suggests that future multi-job schedulers could benefit significantly from exposing internal job dependency graphs.

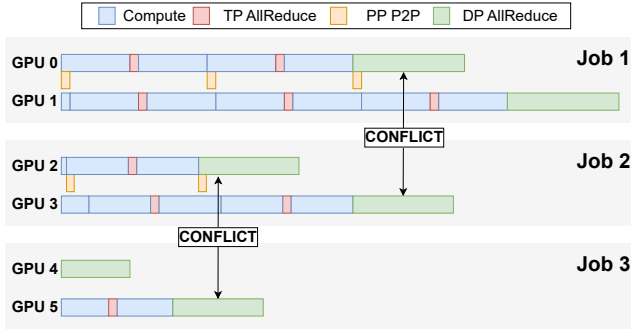


Figure 12: Timeline showing a scenario where jobs conflict such that a non-critical DP AllReduce conflicts with a critical one from another job, thus encourages prioritisation.

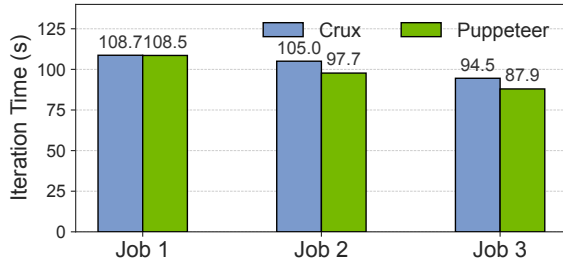


Figure 13: Three jobs on the same network with no compute variation (each job is 90% the size of the last). Puppeteer reclaims time from non-critical communication.

7 Discussion

This section discusses the challenges of utilising Puppeteer in broader deployment scenarios.

Inter-node Mixture-of-Experts. As model sizes continue to grow, the industry is exploring scaling the number of experts [11, 16], necessitating inter-node expert parallelism. This presents a unique challenge for offline planning: the all-to-all traffic pattern in MoE is data-dependent and determined only at runtime by the gating network. This introduces strict latency requirements, as the precise routing matrix is unknown until milliseconds before transmission.

To adapt Puppeteer for inter-node MoE, we envision a hybrid approach involving resource over-provisioning. The planner could allocate bandwidth based on a “worst-case” uniform distribution or historical heuristics, effectively reserving a safe envelope of capacity. At runtime, once the token distribution is finalised, a “refund” mechanism could release unused slots to background transfers. Developing such a low-latency, semi-dynamic refund protocol remains an active area of future research.

Lossy Networks. Our evaluation assumes lossless links, but production networks - especially those spanning multiple data-centres - inevitably experience packet corruption and loss. While Puppeteer’s synchronisation mechanism prevents congestion-induced drops, it does not inherently account for physical layer errors. If a data packet is corrupted, retransmission logic (e.g., Go-Back-N or Selective Repeat [19]) must kick in, effectively behaving like a temporal variance in communication time.

Our existing robustness to jitter handles this well. However, if a *synchronisation handshake* packet is dropped, the dependent GPU could stall indefinitely. To mitigate this, future iterations of Puppeteer could implement a hierarchical control plane: a local scheduler manages intra-DC synchronisation, while a separate, more robust arbiter manages cross-DC traffic with stronger error correction and timeouts.

Link Failure. Physical link failures pose the most severe disruption to a static plan. Since Puppeteer optimises paths based on specific topology availability, a single link loss invalidates the pre-computed routes for all flows assigned to it. We envision three potential recovery strategies:

- (1) **Runtime adaptation.** The enforcement layer could locally detour flows onto backup paths. However, this requires complex orchestration to avoid reintroducing contention and breaking the zero-queue guarantee.
- (2) **On-the-fly replanning.** The central planner could immediately re-compute a new plan for the degraded topology. As shown in Section 6, planning takes only seconds for medium-scale deployments, potentially allowing the training to pause briefly and resume with a fresh optimal plan.
- (3) **Checkpoint recovery.** In the case of catastrophic failure, reverting to the last checkpoint provides ample time for the planner to generate a new schedule offline before the job restarts on the repaired or reconfigured cluster.

8 Conclusion

We presented Puppeteer, a centralised planner that leverages the determinism of AI workloads to replace runtime contention with precise offline orchestration. By treating the network as a statically scheduled resource, Puppeteer achieves a “zero-queue” invariant that existing oblivious protocols cannot guarantee. We hope our work could motivate more research on rearchitecting AI infrastructure to move away from complex in-network logic and toward a “network-as-compiler” model.

Ethics. This work does not raise any ethical issues.

References

- [1] Tommaso Bonato, Abdul Kabbani, Ahmad Ghalayini, Michael Papamichael, Mohammad Dohadwala, Lukas Gianinazzi, Mikhail Khalilov, Elias Achermann, Daniele De Sensi, and Torsten Hoefler. 2025. REPS: Recycled Entropy Packet Spraying for Adaptive Load Balancing and Failure Mitigation. doi:10.48550/arXiv.2407.21625 arXiv:2407.21625 [cs].
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. doi:10.48550/arXiv.2005.14165 arXiv:2005.14165 [cs].
- [3] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2024. Crux: GPU-Efficient Communication Scheduling for Deep Learning Training. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3651890.3672239
- [4] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, Weisen Liu, Yifan Yang, Yichi Xu, Zhilong Zheng, Yu Guan, Kun Qian, Ying Liu, Mingwei Xu, Tianshu Wang, Ning Wang, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. 2025. SyCCL: Exploiting Symmetry for Efficient Collective Communication Scheduling. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 645–662. doi:10.1145/3718958.3750499
- [5] Mosharaf Chowdhury and Ion Stoica. 2012. Coflow: a networking abstraction for cluster applications. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks (HotNets-XI)*. Association for Computing Machinery, New York, NY, USA, 31–36. doi:10.1145/2390231.2390237
- [6] Ofir Cohen, Jose Yallouz Michael Schapira, Shahar Belkar, and Tal Mizrahi. 2025. Routing for Large ML Models. doi:10.48550/arXiv.2503.05324 arXiv:2503.05324 [cs].
- [7] United Nations Regional Information Centre for Western Europe. 2025. Artificial intelligence: How much energy does AI use? <https://unric.org/en/artificial-intelligence-how-much-energy-does-ai-use/> [Accessed: 2026-02-06].
- [8] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 57–70. doi:10.1145/3651890.3672233
- [9] Xinchu Han, Yongxi Lv, Weihao Jiang, Shuyuan Zhang, Yingming Mao, Shizhen Zhao, Zhuoran Liu, Zhuotao Liu, Peirui Cao, Ximeng Liu, and Xinbing Wang. 2024. A Highly Scalable LLM Clusters with Optical Interconnect. <https://arxiv.org/abs/2411.01503v8>
- [10] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. 2018. PipeDream: Fast and Efficient Pipeline Parallel DNN Training. doi:10.48550/arXiv.1806.03377 arXiv:1806.03377 [cs].
- [11] Xu Owen He. 2024. Mixture of A Million Experts. arXiv:2407.04153 [cs.LG] <https://arxiv.org/abs/2407.04153>
- [12] Torsten Hoefler, Karen Schramm, Eric Spada, Keith Underwood, Cedell Alexander, Bob Alverson, Paul Bottorff, Adrian Caulfield, Mark Handley, Cathy Huang, Costin Raiciu, Abdul Kabbani, Eugene Opsasnick, Rong Pan, Adeel Ran, and Rip Sohan. 2025. Ultra Ethernet's Design Principles and Architectural Innovations. doi:10.48550/arXiv.2508.08906 arXiv:2508.08906 [cs].
- [13] Yinsicheng Jiang, Yao Fu, Yeqi Huang, Ping Nie, Zhan Lu, Leyang Xue, Congjie He, Man-Kit Sit, Jilong Xue, Li Dong, Ziming Miao, Dayou Du, Tairan Xu, Kai Zou, Edoardo Ponti, and Luo Mai. 2025. MoE-CAP: Benchmarking Cost, Accuracy and Performance of Sparse Mixture-of-Experts Systems. arXiv:2412.07067 [cs.LG] <https://arxiv.org/abs/2412.07067>
- [14] Mehrdad Khani, Many Ghobadi, Mohammad Alizadeh, Ziyi Zhu, Madeleine Glick, Keren Bergman, Amin Vahdat, Benjamin Klenk, and Eiman Ebrahimi. 2021. SiP-ML: high-bandwidth optical network interconnects for machine learning training. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 657–675. doi:10.1145/3452296.3472900
- [15] Jinkun Lin, Ziheng Jiang, Zuquan Song, Sida Zhao, Menghan Yu, Zhanghan Wang, Chenyuan Wang, Zuocheng Shi, Xiang Shi, Wei Jia, Zherui Liu, Shuguang Wang, Haibin Lin, Xin Liu, Aurojit Panda, and Jinyang Li. 2025. Understanding Stragglers in Large Model Training Using What-if Analysis. doi:10.48550/arXiv.2505.05713 arXiv:2505.05713 [cs].
- [16] Aixin Liu et al. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [17] Tongrui Liu, Chenyang Hei, Fuliang Li, Chengxi Gao, Jiamin Cao, Tianshu Wang, Ennan Zhai, and Xingwei Wang. 2025. ResCCL: Resource-Efficient Scheduling for Collective Communication. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 55–70. doi:10.1145/3718958.3750514
- [18] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. 2024. Rethinking Machine Learning Collective Communication as a Multi-Commodity Flow Problem. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 16–37. doi:10.1145/3651890.3672249
- [19] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. 2018. Revisiting network support for RDMA. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. Association for Computing Machinery, New York, NY, USA, 313–326. doi:10.1145/3230543.3230557
- [20] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (Budapest, Hungary) (SIGCOMM '18)*. Association for Computing Machinery, New York, NY, USA, 221–235. doi:10.1145/3230543.3230564
- [21] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostafa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. doi:10.48550/arXiv.2104.04473 arXiv:2104.04473 [cs].
- [22] NVIDIA. 2025. NVIDIA DOCA PCC Application Guide. <https://docs.nvidia.com/doca/archive/doca-v2.2.0/pcc/index.html> [Accessed: 2026-02-06].
- [23] NVIDIA. 2025. NVIDIA GB200 NVL72. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/> [Accessed: 2026-02-06].
- [24] NVIDIA. 2025. NVIDIA Unveils Rubin CPX: A New Class of GPU Designed for Massive-Context Inference. <https://nvidianews.nvidia.com/news/nvidia-unveils-rubin-cpx->

- a-new-class-of-gpu-designed-for-massive-context-inference [Accessed: 2026-02-06].
- [25] Rui Pan, Yiming Lei, Jialong Li, Zhiqiang Xie, Binhang Yuan, and Yiting Xia. 2022. Efficient flow scheduling in distributed deep learning training with echelon formation. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks (HotNets '22)*. Association for Computing Machinery, New York, NY, USA, 93–100. doi:10.1145/3563766.3564096
 - [26] Jonathan Perry, Amy Ousterhout, Hari Balakrishnan, Devavrat Shah, and Hans Fugal. 2014. Fastpass: a centralized "zero-queue" datacenter network. *SIGCOMM Comput. Commun. Rev.* 44, 4 (Aug. 2014), 307–318. doi:10.1145/2740070.2626309
 - [27] Konstantinos Prasopoulos, Ryan Kosta, Edouard Bugnion, and Marios Kogias. 2025. SIRD: A Sender-Informed, Receiver-Driven Datacenter Transport Protocol. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 451–471. <https://www.usenix.org/conference/nsdi25/presentation/prasopoulos>
 - [28] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. {CASSINI}: {Network-Aware} Job Scheduling in Machine Learning Clusters. 1403–1420. <https://www.usenix.org/conference/nsdi24/presentation/rajasekaran>
 - [29] Sudarsanan Rajasekaran, Sanjoli Narang, Anton A. Zabreyko, and Manya Ghobadi. 2024. MLTCP: A Distributed Technique to Approximate Centralized Flow Scheduling For Machine Learning. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 167–176. doi:10.1145/3696348.3696878
 - [30] Sundararajan Renganathan and Nick McKeown. 2025. Chronos: Prescheduled circuit switching for LLM training. In *Proceedings of the 2nd Workshop on Networks for AI Computing (NAIC '25)*. Association for Computing Machinery, New York, NY, USA, 89–97. doi:10.1145/3748273.3749210
 - [31] Adel Sefiane, Alireza Farshin, and Marios Kogias. 2025. MLSynth: Towards Synthetic ML Traces. In *Proceedings of the 2nd Workshop on Networks for AI Computing (NAIC '25)*. Association for Computing Machinery, New York, NY, USA, 98–104. doi:10.1145/3748273.3749211
 - [32] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. {TACCL}: Guiding Collective Algorithm Synthesis using Communication Sketches. 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
 - [33] Srinivas Sridharan, Taekyung Heo, Louis Feng, Zhaodong Wang, Matt Bergeron, Wenyin Fu, Shengbao Zheng, Brian Coutinho, Saeed Rashidi, Changhai Man, and Tushar Krishna. 2023. Chakra: Advancing Performance Benchmarking and Co-design using Standardized Execution Traces. doi:10.48550/arXiv.2305.14516 arXiv:2305.14516 [cs].
 - [34] Xincheng Wan, Xinyu Yang, Kaiqiang Xu, Xudong Liao, Yilun Jin, Yijun Sun, Zhenghang Ren, Han Tian, and Kai Chen. 2025. Coflow Scheduling for LLM Training. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 1232–1234. doi:10.1145/3718958.3750467
 - [35] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, Yang Liu, Pengcheng Zhang, Kun Qian, Kunling He, Jiaqi Gao, Ennan Zhai, Dennis Cai, and Binzhang Fu. 2025. {SimAI}: Unifying Architecture Design and Performance Tuning for {Large-Scale} Large Language Model Training with Scalability and Precision. 541–558. <https://www.usenix.org/conference/nsdi25/presentation/wang-xizheng-simai>
 - [36] xAI. 2025. Colossus. <https://x.ai/colossus> [Accessed: 2026-02-06].
 - [37] Yunhong Xu, Keqiang He, Rui Wang, Minlan Yu, Nick Duffield, Hassan Wassel, Shidong Zhang, Leon Poutievski, Junlan Zhou, and Amin Vahdat. 2022. Hashing Design in Modern Networks: Challenges and Mitigation Techniques. 805–818. <https://www.usenix.org/conference/atc22/presentation/xu>
 - [38] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. 2025. White-Boxing RDMA with Packet-Granular Software Control. 427–449. <https://www.usenix.org/conference/nsdi25/presentation/zhao-chenxingyu>
 - [39] Boyang Zhou, Chunming Wu, Qiang Yang, and Bing Hu. 2025. Planning ECMP Paths with Minimal Overlap for Efficient Cross-Host Collective Communications. In *2025 IEEE 31th International Conference on Parallel and Distributed Systems (ICPADS)*. 1–10. doi:10.1109/ICPADS67057.2025.11323027 ISSN: 2690-5965.